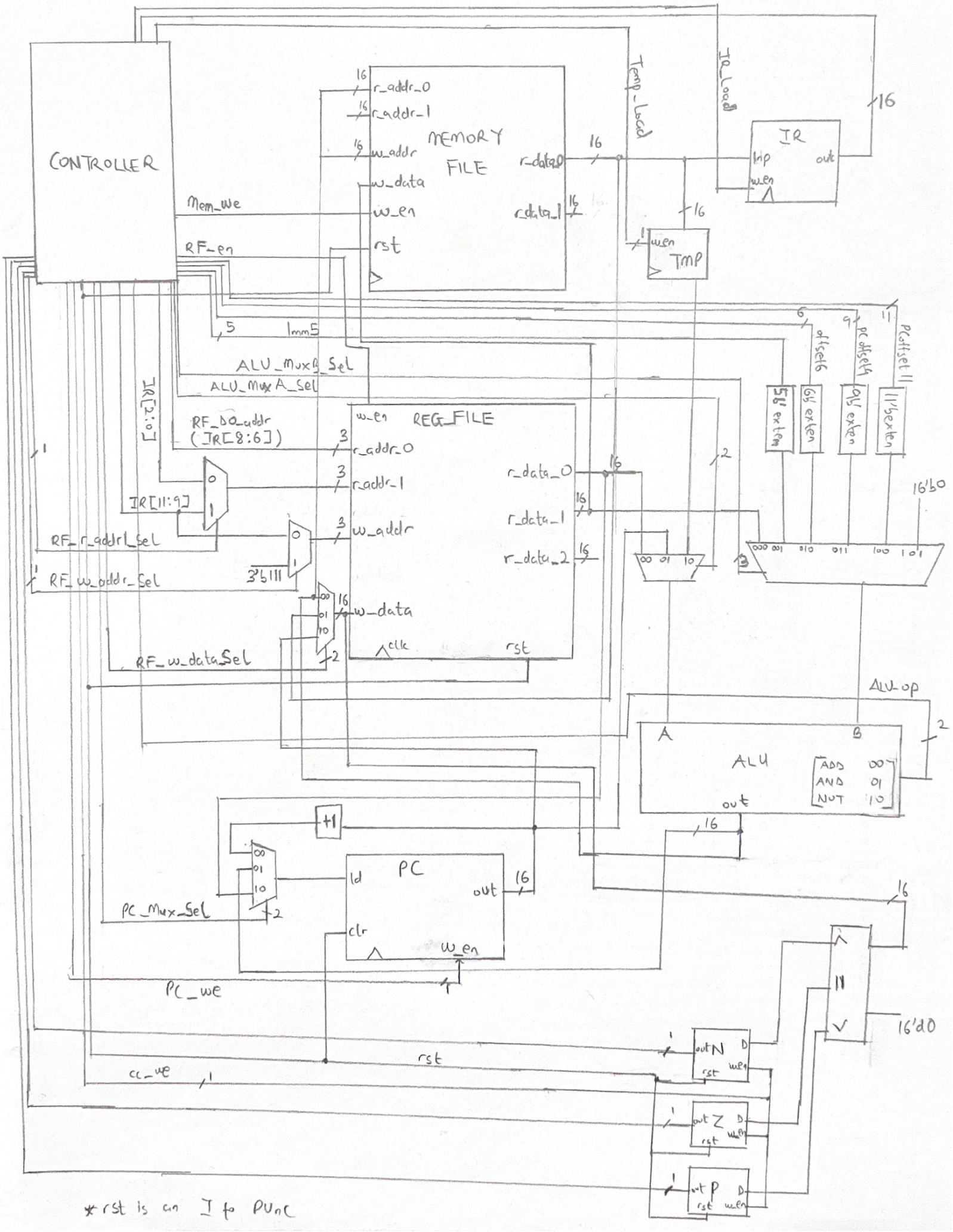
Datapath drawing



Control Signals	FETCH	DECODE	ADD1	ADD2	AND1	AND2	NOT	BR	JMP	JSR	JSRR	LD	LDI (cyc.1)	LDI (cyc.2)	LDR	LEA	RET	ST	STI(cyc.1)	STI(cyc.2)	STR	HALT
PC_Mux_Sel	X	2'b00	X	X	X	X	X	LOGIC	2'b10	2'b01	2'b10	X	X	X	X	X	2'b10	X	X	X	X	X
ALU_MuxA_Sel	2'b01	X	2'b00	2'b00	2'b00	2'b00	2'b00	2'b01	X	2'b01	X	2'b01	2'b01	2'b10	2'b01	2'b01	X	2'b01	2'b01	2'b10	2'b00	X
ALU_MuxB_Sel	3'b101	X	3'b000	3'b001	3'b000	3'b001	X	3'b011	X	3'b100	X	3'b011	3'b011	3'b101	3'b010	3'b011	X	3'b011	3'b011	3'b101	3'b010	X
ALU_Op	2'b00	X	2'b00	2'b00	2'b01	2'b01	2'b10	2'b00	X	2'b00	X	2'b00	2'b00	2'b00	2'b00	2'b00	X	2'b00	2'b00	2'b00	2'b00	X
RF_w_data_Sel	X	X	2'b00	2'b00	2'b00	2'b00	2'b00	X	X	2'b10	2'b10	2'b01	X	2'b01	2'b01	2'b01	X	X	X	X	X	X
RF_w_addr_Sel	X	X	1'b0	1'b0	1'b0	1'b0	1'b0	X	X	1'b1	1'b1	1'b0	X	1'b0	1'b0	1'b0	X	X	X	X	X	X
RF_r_addr1_Sel	X	X	1'b0	X	1'b0	X	X	X	X	X	X	X	X	X	X	X	X	1'b1	X	1'b1	1'b1	X
RF_we	0	0	1	1	1	1	1	0	0	1	1	1	0	1	1	1	0	0	0	0	0	0
Mem_we	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	0
IR_Load	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Temp_Load	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0
CC_we	0	0	1	1	1	1	1	0	0	0	0	1	0	0	1	1	0	0	0	0	0	0
PC_we	0	1	0	0	0	0	0	LOGIC	1	1	1	0	0	0	0	0	1	0	0	0	0	0

ANNOTATED, COMMENTED COPY OF ASSEMBLY CODE

```
================================================================================
// Program Name: Array Maximum Finder
// Description:  Traverses an array of 5 integers and stores the largest value.
================================================================================

// --- INITIALIZATION BLOCK ---
/* 00: */ LEA R0, #17     ; R0 = Base Address of Data Array (PC+1+17 = Address 18)
/* 01: */ LD  R1, #14     ; R1 = Loop Counter (Load "5" from Address 16)
/* 02: */ AND R2, R2, #0  ; R2 = 0 (Initialize Max Value to 0)

// --- MAIN LOOP ---
/* 03: */ LDR R3, R0, #0  ; R3 = MEM[R0 + 0] (Load current array value)
/* 04: */ JSR #5          ; Jump to Subroutine CHECK_MAX (PC+1+5 = Address 10)
/* 05: */ ADD R0, R0, #1  ; Increment Array Pointer (Move to next element)
/* 06: */ ADD R1, R1, #-1 ; Decrement Loop Counter
/* 07: */ BRp #-5         ; If Counter > 0, Branch back to LOOP (PC+1-5 = Address 3)

// --- TERMINATION ---
/* 08: */ ST  R2, #8      ; Store Result (Max Value) to Address 17 (PC+1+8)
/* 09: */ // HALT          ; Stop the Processor

// --- SUBROUTINE: CHECK_MAX ---
// Inputs: R3 (New Value), R2 (Current Max)
// Output: R2 (Updated Max)
// Logic:  If (Value - Max) > 0, Then Max = Value
/* 10: */ NOT R4, R2      ; R4 = NOT(Max)
/* 11: */ ADD R4, R4, #1  ; R4 = -Max (Two's Complement)
/* 12: */ ADD R4, R3, R4  ; R4 = Value + (-Max)
```

```
/* 13: */ BRnz #1         ; If (Value - Max) <= 0, Skip update

/* 14: */ ADD R2, R3, #0  ; Update Max: R2 = Value

/* 15: */ RET             ; Return to Caller (PC = R7)


// --- DATA SEGMENT ---

/* 16: */ 0005  // CONST: Loop Count = 5

/* 17: */ 0000  // VAR:   Result (Final Max stored here)

/* 18: */ 0005  // ARR[0]

/* 19: */ 0002  // ARR[1]

/* 20: */ 0008  // ARR[2] (Expected Max)

/* 21: */ 0001  // ARR[3]

/* 22: */ 0004  // ARR[4]
```

Explanation of Assembly Program:

The program demonstrates the capabilities of PUnC by iterating through a sequential array of integers stored in memory to identify and isolate the largest value. This program was chosen because it requires a comprehensive subset of the LCT-3 instruction set, utilizing 11 unique instructions (*LEA, LD, AND, LDR, JSR, ADD, BR, ST, HALT, NOT, RET*) to perform pointer arithmetic, modular logic and memory manipulation.

Detailed Operation The program begins by initializing pointers and variables. It uses LEA to load the starting address of the data array into register R0 and LD to load the loop counter (5) into R1. The maximum value register (R2) is initialized to zero using the AND instruction with immediate 0.

The core logic operates inside a loop that performs the following steps 5 times:

1. **Load:** The program uses LDR to fetch the current integer from the array based on the pointer in R0.
2. **Compare (Subroutine):** The program uses JSR to jump to a modular subroutine named CHECK_MAX. Inside this subroutine, 2's complement arithmetic (NOT and ADD) is used to subtract the Current Max from the New Value.
    - If the result is positive (BRp logic), the New Value is larger, and R2 is updated.
    - The RET instruction restores the Program Counter to the main loop.
3. **Update:** The array pointer (R0) is incremented, and the loop counter (R1) is decremented using ADD.

4. **Loop:** The BRp instruction checks if the counter is still positive. If so, it branches back to the start of the loop.

Once the loop completes, the program stores the final maximum value into a designated memory location using ST and terminates execution with HALT. In the test case provided, the program successfully identifies 8 as the maximum value from the array {5, 2, 8, 1, 4}.

Feedback:

This lab was very interesting especially getting to see the PUnC finally working and executing some programs. The main issue for me was getting the Datapath and Control signals table right; this took the bulk of my time since it defines the entire project, but after getting it done, the rest was fairly straightforward. The checkpoints were very helpful in helping me break the entire lab into chunks and being on track to finish it. Overall, it was quite challenging (especially with the design and debugging) but also very rewarding.